

BAŞLIK: API YÖNETİMİ, ÖLÇEKLENDİRME VE GERÇEK DÜNYA ÖRNEKLERİ

TÜR: Akademik/Teknik Detaylı Anlatım

BÖLÜM 1: TEK API'DEN MİKROSERVİSLERE

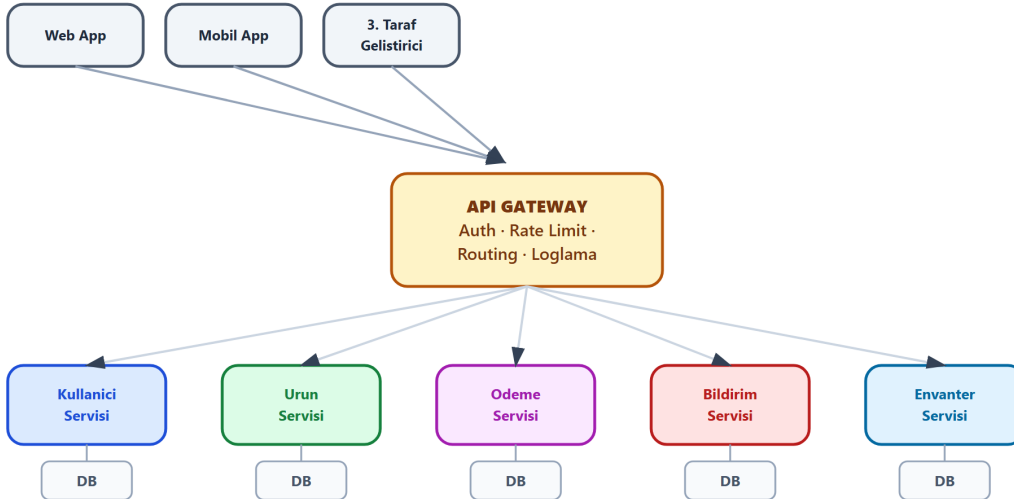
22.6'da inşa ettiğimiz Kütüphane API'si gibi, tek bir uygulamanın (monolith) TÜM işlevlerini (kullanıcılar, kitaplar, ödünç alma, bildirimler) TEK bir kod tabanında ve TEK bir API'de barındırdığı mimariye MONOLİTİK mimari denir. Küçük/orta ölçekli projeler için bu YETERLİ ve hatta TERCİH EDİLEN bir yaklaşımdır — daha basittir, daha az operasyonel karmaşıklık taşır. Ama bir şirket büyüdükçe (onlarca mühendis, milyonlarca kullanıcı), monolitik yapı sorun çıkarmaya başlar: TEK bir bölümdeki (örn. bildirim gönderme) bir hata TÜM sistemi çökertebilir, TEK bir küçük değişiklik için TÜM devasa uygulamanın yeniden deploy edilmesi gerekir, farklı bölümlerin farklı hızlarda ölçeklenmesi (örn. "ürün arama" trafiği "ödeme"den 100 kat fazla olabilir) mümkün olmaz.

MİKROSERVİS mimarisi, bu monolitik yapıyı, her biri BAĞIMSIZ deploy edilebilen, kendi veritabanına sahip, küçük, odaklanmış servislere böler (Kullanıcı Servisi, Ürün Servisi, Ödeme Servisi...). Her servis kendi API'sini sunar ve servisler birbirleriyle YİNE API'ler (genellikle REST ya da gRPC, bkz. 22.8) üzerinden konuşur. Bu, ekiplerin bağımsız çalışmasını (bkz. 22.1, Bölüm 2.4) mimari seviyeye taşır.

BÖLÜM 2: API GATEWAY DESENİ

Mikroservis mimarisinin doğal bir sorunu vardır: eğer her istemci (web, mobil) HER mikroservise DOĞRUDAN bağlanmak zorunda kalırsa, (1) istemci kodu, hangi işlevin hangi sunucuda olduğunu bilmek zorunda kalır (sıkı bağlılık - tight coupling), (2) kimlik doğrulama, hız sınırlama gibi ORTAK mantığın HER SERVİSTE ayrı ayrı yazılması gerekir (kod tekrarı). API Gateway deseni, tüm istemci trafiğinin geçtiği TEK bir giriş noktası (kapı) tanımlayarak bu sorunu çözer:

API Gateway ve Mikroservis Mimarisi



İstemciler tek bir kapiyi (gateway) görür; arka plandaki servisler bağımsız ölçek/deploy edilir.

Şekil 1 — API Gateway, tüm ortak sorumlulukları (kimlik doğrulama, hız sınırlama, yönlendirme, loglama) tek bir katmanda merkezileştirir.

API Gateway'in tipik sorumlulukları: (1) Yönlendirme (Routing) — gelen isteği doğru mikroservise iletmek; (2) Kimlik Doğrulama — token'ı BİR KERE, merkezi olarak doğrulamak (bkz. 22.4); (3) Hız Sınırlama — tüm sisteme giren trafiği merkezi olarak kontrol etmek (bkz. 22.4, Bölüm 3); (4) Loglama/İzleme (bkz. Bölüm 3); (5) Bazen: yanıtları birleştirme (aggregation) — bir istemci isteğinin arka planda birden fazla mikroservise dağılıp, TEK bir yanıt olarak birleştirilmesi.

BÖLÜM 3: İZLENEBİLİRLİK (OBSERVABILITY)

Bir API canlıya alındıktan sonra, "çalışıyor mu?" sorusunun cevabı ARTIK VARSAYIM değil, ÖLÇÜMLE verilmelidir. İzlenebilirlik, bir sistemin İÇ DURUMUNU, dışarıdan gözlemlenebilir sinyallerden (loglar, metrikler, izler) anlayabilme yeteneğidir ve 3 ana bileşenden oluşur:

1. Loglama (Logging): Her isteğin ne zaman, hangi kullanıcı tarafından, hangi sonuçla (durum kodu, süre) işlendiğinin kaydı. Hata ayıklama (debugging) için birincil kaynaktır.
2. Metrikler (Metrics): Zaman içinde toplanan sayısal veriler — saniyedeki istek sayısı (RPS), ortalama/95. yüzdelik (p95) yanıt süresi, hata oranı (%). Bu metrikler genellikle Grafana gibi araçlarla panolarda (dashboard) görselleştirilir ve belirli eşikler aşıldığında otomatik uyarılar (alert) tetiklenir.
3. Dağıtık İzleme (Distributed Tracing): Bir isteğin, API Gateway'den girip kaç farklı mikroservisten geçtiğini VE HER birinde ne kadar süre harcadığını uçtan uca takip eder — mikroservis mimarilerinde "neden bu istek 3 saniye sürdü?" sorusuna cevap vermenin tek yoludur.

3.1. Hizmet Seviyesi Anlaşmaları (SLA/SLO)

Büyük API sağlayıcıları, müşterilerine belirli bir güvenilirlik SÖZ VERİR — örneğin "API'miz ayda %99.9 çalışır durumda olacak" (bu, ayda yaklaşık 43 dakikadan fazla kesinti olmayacağı anlamına gelir). Bu hedefe SLO (Service Level Objective) denir; müşteriyle yapılan resmi sözleşmeye ise SLA (Service Level Agreement) denir ve genellikle hedefin altına düşülürse müşteriye tazminat (kredi) verilmesini içerir.

BÖLÜM 4: GERÇEK DÜNYA VAKA ÇALIŞMALARI

4.1. Stripe: Geliştirici Deneyimini Ürün Stratejisi Yapmak

Stripe, kuruluşundan itibaren API tasarımını (tutarlı isimlendirme, mükemmel dokümantasyon, kapsamlı test/sahte - mock - ortamı) BİRİNCİL rekabet avantajı olarak konumlandırdı. Stripe API'si genellikle "en iyi tasarlanmış API" örneği olarak gösterilir: her kaynak tutarlı şekilde isimlendirilir, HER hata anlamlı bir kod ve insan tarafından okunabilir bir mesajla döner, sürümleme (bkz. 22.3, Bölüm 5) yıllarca geriye dönük UYUMLULUĞU garanti eder. Sonuç: geliştiriciler Stripe'ı "kullanması keyifli" bulur ve bu, doğrudan şirketin büyümesine yansımıştır.

4.2. Twitter/X API v1.1 → v2 Geçişi: Sürümleme Zorlukları

Twitter'ın API'sini v1.1'den v2'ye taşıması, kötü yönetilen bir API sürüm geçişinin klasik örneklerinden biri olarak literatürde sıkça anılır: uzun süre iki sürüm paralel yaşadı, bazı işlevler v2'ye HİÇ taşınmadı, fiyatlandırma politikası aniden ve sert şekilde değişti. Bu, binlerce üçüncü parti uygulamanın (bkz. 22.1, Bölüm 4) aniden bozulmasına ya da iş modellerinin çökmesine yol açtı. Ders: bir API'nin sürüm geçiş stratejisi (bkz. 22.3, Bölüm 5), sadece teknik değil aynı zamanda GÜVEN inşa eden bir iş kararıdır — geliştiricilere yeterli geçiş süresi tanımamak, o API etrafında kurulmuş TÜM ekosistemi riske atar.

4.3. GitHub API: Hem REST Hem GraphQL Sunmak

GitHub, hem klasik bir REST API (v3) hem de bir GraphQL API (v4) sunarak, bu modülde öğrendiğimiz "her mimarinin kendi güçlü olduğu senaryo vardır" (bkz. 22.8) prensibini gerçek dünyada birebir uygular: basit, tek kaynaklı işlemler için REST; bir kullanıcının profilini, repolarını, issue'larını TEK istekte, TAM istenen alanlarla çekmek gibi karmaşık, iç içe sorgular için GraphQL tercih edilebilir.

SONUÇ VE MODÜLÜN TAMAMLANMASI

Monolitik mimariden mikroservislere geçişi, API Gateway desenini, izlenebilirliğin 3 sütununu (log, metrik, iz) ve SLA/SLO kavramlarını öğrendik; ardından Stripe, Twitter/X ve GitHub örnekleriyle bu teorinin gerçek dünyada nasıl iş sonuçlarına dönüştüğünü gördük.

Bu modülde, bir API'nin ne olduğundan (22.1) başlayıp, taşıma protokolüne (22.2), tasarım prensiplerine (22.3), güvenliğine (22.4), Python ile hem tüketilmesine (22.5) hem inşa edilmesine (22.6), dokümantasyon/test pratiklerine (22.7), alternatif mimarilere (22.8) ve son olarak production'da yönetimine (22.9) kadar uçtan uca bir yolculuk yaptık. Artık hem VAR OLAN bir API'yi güvenle, dayanıklı bir şekilde kullanabilecek hem de KENDİ API'nizi, endüstri standartlarında, güvenli ve iyi dokümante edilmiş şekilde inşa edebilecek bilgiye sahipsiniz. Modülü, 22.10'daki genel tekrar sorularıyla pekiştirebilirsiniz.